

Deploying NexHRIS on a Hostinger VPS

(KVM 1)

A complete walkthrough, from an empty VPS to a working system on your own domain. Follow it top to bottom. Nothing is assumed to be done already.

What you are building

Piece	Choice	Why
Operating system	Ubuntu 24.04 LTS	Long-term support, and every command below is written for it
Web server	Nginx	Lighter than Apache on 1 vCPU
PHP	8.3 FPM	The system needs 8.2 or newer
Database	MariaDB	Drop-in for MySQL, lighter
PDF conversion	LibreOffice	Converts the Civil Service forms exactly as the campus laid them out
Certificate	Let's Encrypt (Certbot)	Free, renews itself

KVM 1 gives you 1 vCPU, 4 GB RAM and 50 GB of disk. That is comfortable for this system. LibreOffice is the heaviest thing on the box and it only runs while somebody is converting a form.

Conventions used below

- `nexhris.example.ph` — replace with your real domain, every time it appears.
- `deploy` — the Linux user you will create. Never run the site as `root`.
- Lines beginning with `$` are typed by you. Do not type the `$`.
- Anything in `CAPITALS_LIKE_THIS` is a value you choose and must write down.

Keep a text file open as you go and paste every password into it. You will need them again in Part 5.

Part 0 — Before you touch the VPS

These two things are done on your Windows machine, not the server. Skipping them is the most common way a deployment "works" but ships the wrong code.

0.1 Commit your work

Your project has uncommitted changes. If you deploy by pulling from Git and these are not committed, the server gets an older system.

```
cd /c/xampp/htdocs/nexhris
git status                # look at what is listed
git add -A
git commit -m "Prepare for production deployment"
git push origin main
```

If `git push` complains that there is no remote, create an empty **private** repository on GitHub first, then:

```
git remote add origin https://github.com/YOUR-USERNAME/nexhris.git
git push -u origin main
```

Keep the repository **private**. It does not contain passwords — `.env` is ignored — but it is your capstone.

No GitHub? You can upload the folder with SFTP instead. See Appendix B. Git is strongly preferred because updating later becomes one command.

0.2 Decide these values now

Write them down. You will paste them in later.

Value	Example	Yours
Domain	nexhris.example.ph	
Server user password	a long random phrase	
Database name	nexhris	
Database user	nexhris_user	
Database password	a long random phrase	
HR Administrator email	hr.tagudin@ispsec.edu.ph	
Backup password	a long random phrase	

Part 1 — Create and secure the VPS

1.1 Order the plan

1. Log in to Hostinger and open **VPS → KVM 1**.
2. When asked for an operating system, choose **Ubuntu 24.04 LTS**, plain — not a template with a control panel. A panel will fight you over Nginx later.
3. Set the root password when prompted and save it.
4. Wait for the status to become **Running**, then copy the **IP address** shown on the VPS overview page.

1.2 Connect

From Windows, open PowerShell or Git Bash:

```
ssh root@YOUR_SERVER_IP
```

Type `yes` at the fingerprint question, then the root password.

Hostinger also has a **Browser terminal** button on the VPS page. If SSH gives you trouble, use that — every command below works the same way there.

1.3 Update the system

```
apt update && apt upgrade -y
```

If it asks about keeping a configuration file, accept the default by pressing Enter. If it asks to restart services, choose **Ok**.

1.4 Create a non-root user

Running a web app as root means any flaw in it is a flaw with full control of the machine.

```
adduser deploy
```

Give it a password when asked. Press Enter through the name and phone questions. Then grant it administrative rights:

```
usermod -aG sudo deploy
```

Let it use the same SSH key or password you just used for root:

```
rsync --archive --chown=deploy:deploy ~/.ssh /home/deploy 2>/dev/null || true
```

Now log out and back in as `deploy`:

```
exit  
ssh deploy@YOUR_SERVER_IP
```

From here on, every command is run as `deploy`. Commands that need administrative rights start with `sudo`, which will ask for the `deploy` password the first time.

1.5 Turn on the firewall

```
sudo ufw allow OpenSSH  
sudo ufw allow 'Nginx Full'  
sudo ufw enable
```

Type `y` to confirm. Check it:

```
sudo ufw status
```

You should see `22`, `80` and `443` allowed and nothing else.

Part 2 — Point your domain at the server

Do this now, because the certificate in Part 7 cannot be issued until the domain resolves, and DNS takes time to spread.

1. Open wherever your domain is registered (Hostinger's **Domains** section, or another registrar).
2. Go to **DNS / Nameservers** → **Manage DNS records**.
3. Add or edit an **A record**:
 - **Type**: A
 - **Name**: @ (this means the domain itself)
 - **Points to**: YOUR_SERVER_IP
 - **TTL**: leave the default
4. Add a second **A record** for **www**:
 - **Type**: A, **Name**: www, **Points to**: YOUR_SERVER_IP

Check it from the server. It can take from a few minutes to a few hours:

```
dig +short nexhris.example.ph
```

When that prints your server's IP, DNS is ready. Continue with the other parts while you wait — only Part 7 actually needs it.

Part 3 — Install the software

3.1 Nginx

```
sudo apt install -y nginx
```

Visit `http://YOUR_SERVER_IP` in a browser. You should see "Welcome to nginx". If you do not, the firewall rule in 1.5 did not apply — re-run it.

3.2 PHP 8.3 and its extensions

Ubuntu 24.04 ships PHP 8.3, which satisfies the system's requirement of 8.2 or newer.

```
sudo apt install -y php8.3-fpm php8.3-cli php8.3-mysql php8.3-mbstring \
php8.3-xml php8.3-curl php8.3-zip php8.3-gd php8.3-bcmath php8.3-intl
```

Every one of those is needed:

Extension	Used for
<code>mysql</code>	the database
<code>zip</code>	reading and writing <code>.xlsx</code> workbooks
<code>gd</code>	profile photos and the digital ID QR code
<code>mbstring</code> , <code>xml</code>	the spreadsheet and PDF libraries
<code>curl</code>	outbound requests
<code>bcmath</code> , <code>intl</code>	leave credit arithmetic and date formatting

Confirm:

```
php -v # must say 8.2 or 8.3
php -m | grep -E 'zip|gd|mbstring|curl|mysql'
```

3.3 MariaDB

```
sudo apt install -y mariadb-server
sudo mysql_secure_installation
```

Answer the prompts:

- *Enter current password for root:* press **Enter** (there is none yet)
- *Switch to unix_socket authentication:* **n**
- *Change the root password:* **Y**, then set one and write it down
- *Remove anonymous users:* **Y**
- *Disallow root login remotely:* **Y**
- *Remove test database:* **Y**
- *Reload privilege tables:* **Y**

3.4 LibreOffice

This is what turns the uploaded Civil Service workbooks into PDFs that match the campus's own forms.

```
sudo apt install -y libreoffice-calc libreoffice-writer fonts-liberation
```

`libreoffice-writer` is needed because page two of the leave form is an embedded Word document. `fonts-liberation` provides metric-compatible Arial and Times, so the forms keep their spacing.

Confirm it is there and note the path:

```
which soffice          # expect /usr/bin/soffice
soffice --version
```

The first `soffice --version` can take 10–20 seconds. That is normal; it is building its profile.

3.5 Composer

```
cd ~
curl -sS https://getcomposer.org/installer -o composer-setup.php
sudo php composer-setup.php --install-dir=/usr/local/bin --filename=composer
rm composer-setup.php
composer --version
```

3.6 Node.js 20

Needed to build the stylesheet and JavaScript once.

```
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -  
sudo apt install -y nodejs  
node -v # expect v20.x
```

3.7 Git and unzip

```
sudo apt install -y git unzip
```

Part 4 — Create the database

```
sudo mysql -u root -p
```

Enter the MariaDB root password from 3.3. At the `MariaDB [(none)]>` prompt, type each line exactly, including the semicolons, replacing `YOUR_DB_PASSWORD`:

```
CREATE DATABASE nexhris CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;  
CREATE USER 'nexhris_user'@'localhost' IDENTIFIED BY 'YOUR_DB_PASSWORD';  
GRANT ALL PRIVILEGES ON nexhris.* TO 'nexhris_user'@'localhost';  
FLUSH PRIVILEGES;  
EXIT;
```

Check that the new user works:

```
mysql -u nexhris_user -p nexhris -e "SELECT 'connected';"
```

Part 5 — Put the system on the server

5.1 Fetch the code

```
sudo mkdir -p /var/www
sudo chown deploy:deploy /var/www
cd /var/www
git clone https://github.com/YOUR-USERNAME/nexhris.git
cd nexhris
```

If the repository is private, GitHub will ask for a username and password. GitHub no longer accepts your account password here — create a **personal access token** at github.com → Settings → Developer settings → Personal access tokens → Tokens (classic), tick `repo`, and paste the token as the password.

Not using Git? See Appendix B, then come back here at 5.2.

5.2 Install PHP dependencies

```
composer install --no-dev --optimize-autoloader
```

`--no-dev` leaves out the testing tools, which production does not need. Note this for later: it also removes PHPUnit, so `php artisan test` will not run on the server. That is expected.

5.3 Build the stylesheet and JavaScript

```
npm ci
npm run build
```

This writes `public/build`. It takes a minute or two on 1 vCPU. When it is done you can free the space:

```
rm -rf node_modules
```

5.4 Write the production `.env`

```
cp .env.example .env
nano .env
```

`nano` is a plain text editor. Arrow keys move, typing replaces, `Ctrl+O` then Enter saves, `Ctrl+X` exits.

Set these. Everything else in the file can stay as it is.

```
APP_NAME="NexHRIS"
APP_ENV=production
APP_DEBUG=false
APP_URL=https://nexhris.example.ph

DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=nexhris
DB_USERNAME=nexhris_user
DB_PASSWORD=YOUR_DB_PASSWORD

SESSION_DRIVER=database
SESSION_SECURE_COOKIE=true
CACHE_STORE=database
QUEUE_CONNECTION=database
FILESYSTEM_DISK=local

PDF_RENDERER=auto
LIBREOFFICE_PATH=/usr/bin/soffice

# Leave two-factor OFF until you have proved mail works in Part 9.
TWO_FACTOR_ENABLED=false

BACKUP_PASSWORD=YOUR_BACKUP_PASSWORD
```

`APP_DEBUG=false` is not optional. With it on, any error shows a stack trace containing your database password to whoever triggered it.

Mail settings come in Part 9. Leave them alone for now.

5.5 Generate the application key

```
php artisan key:generate
```

This writes `APP_KEY` into `.env`. It encrypts session cookies and, if you do not set `BACKUP_PASSWORD`, your backups too. **Never change it after go-live** — doing so logs everyone out and makes old backups unreadable.

5.6 Create the database tables

```
php artisan migrate --force
```

`--force` is required because this is a production environment; without it Laravel refuses to migrate. This also creates the campus's colleges and departments, so the organisational structure is ready.

5.7 Create the first HR Administrator

```
php artisan db:seed --force
```

This creates one account:

- **Email:** `admin@ispssc.edu.ph`
- **Password:** `ChangeMe123!`

Change that password the first time you sign in. It is written in the source code, which means it is public.

5.8 Link the public storage folder

```
php artisan storage:link
```

Without this, the blank Civil Service forms and profile photos will not load. Uploaded Personal Data Sheets and leave forms are deliberately *not* in this folder — they live outside the web root and are only released through the application after it checks who is asking.

5.9 Set file ownership and permissions

Nginx and PHP run as the user `www-data`. It needs to write to exactly two places and read everything else.

```
sudo chown -R deploy:www-data /var/www/nexhris
sudo find /var/www/nexhris -type f -exec chmod 644 {} \;
sudo find /var/www/nexhris -type d -exec chmod 755 {} \;
sudo chmod -R 775 /var/www/nexhris/storage /var/www/nexhris/bootstrap/cache
sudo chmod 640 /var/www/nexhris/.env
```

The last line matters: `.env` holds your database password and should not be world-readable.

Part 6 — Configure Nginx and PHP

6.1 Tune PHP

```
sudo nano /etc/php/8.3/fpm/php.ini
```

Use **Ctrl+W** to search for each setting and change it:

```
memory_limit = 512M
upload_max_filesize = 12M
post_max_size = 12M
max_execution_time = 180
```

Why these values:

- **512M** — converting a four-page Personal Data Sheet holds the whole workbook in memory.
- **12M** — the system accepts workbooks up to 10 MB, and the request needs headroom above that.
- **180s** — LibreOffice gives up on its own after 120 seconds, so PHP must not give up first.

Save and exit.

6.2 Confirm **proc_open** is available

The system starts LibreOffice with **proc_open**. If it is disabled, conversions fail.

```
php -r "echo function_exists('proc_open') ? 'proc_open OK' : 'DISABLED', PHP_EOL;"
```

If it says DISABLED, search **disable_functions** in **php.ini** and remove **proc_open** from that list.

6.3 Size PHP-FPM for 1 vCPU

```
sudo nano /etc/php/8.3/fpm/pool.d/www.conf
```

Find and set:

```
pm = ondemand
pm.max_children = 10
pm.process_idle_timeout = 30s
```

```
request_terminate_timeout = 180
```

ondemand keeps memory free when nobody is using the system, which suits a campus that is busy in bursts.

```
sudo systemctl restart php8.3-fpm
```

6.4 The Nginx site

```
sudo nano /etc/nginx/sites-available/nexhris
```

Paste this, changing only the two **server_name** occurrences:

```
server {
    listen 80;
    listen [::]:80;
    server_name nexhris.example.ph www.nexhris.example.ph;

    # Laravel serves from public/, never from the project root. Pointing at
    # the root would expose .env and every uploaded record.
    root /var/www/nexhris/public;
    index index.php;

    charset utf-8;
    client_max_body_size 12M;

    location / {
        try_files $uri $uri/ /index.php?$query_string;
    }

    location ~ \.php$ {
        include snippets/fastcgi-php.conf;
        fastcgi_pass unix:/run/php/php8.3-fpm.sock;

        # A form conversion can take over a minute on one core.
        fastcgi_read_timeout 180;
    }

    # Never serve dotfiles, and never serve .env whatever happens above.
    location ~ /\.(!well-known).* {
        deny all;
    }
}
```

```
location = /favicon.ico { access_log off; log_not_found off; }  
location = /robots.txt { access_log off; log_not_found off; }  
  
error_log /var/log/nginx/nexhris-error.log;  
access_log /var/log/nginx/nexhris-access.log;  
}
```

Enable it, remove Nginx's default site, and check the syntax:

```
sudo ln -s /etc/nginx/sites-available/nexhris /etc/nginx/sites-enabled/  
sudo rm /etc/nginx/sites-enabled/default  
sudo nginx -t  
sudo systemctl reload nginx
```

`nginx -t` must say "syntax is ok" and "test is successful". If it does not, re-read the file for a missing `;` or `}`.

6.5 First look

Visit `http://nexhris.example.ph`. You should see the NexHRIS sign-in screen.

If you get **502 Bad Gateway**, the PHP socket path is wrong:

```
ls /run/php/          # confirm the exact .sock filename and fix fastcgi_pass
```

If you get **500** or a blank page, look at the reason:

```
tail -50 /var/www/nexhris/storage/logs/laravel.log
```

Do not continue until the sign-in screen loads.

Part 7 — HTTPS

A personnel system must not run over plain HTTP. The sign-in form would send passwords in the clear.

```
sudo apt install -y certbot python3-certbot-nginx
sudo certbot --nginx -d nexhris.example.ph -d www.nexhris.example.ph
```

Answer the prompts:

- **Email:** yours, for expiry warnings
- **Terms:**
- **Share your email:**
- **Redirect HTTP to HTTPS:** choose **2 (Redirect)**

Certbot rewrites the Nginx file for you and installs a renewal timer. Confirm renewal works:

```
sudo certbot renew --dry-run
```

Now visit <https://nexhris.example.ph>. You should see a padlock.

Part 8 — Cache the configuration

Only now, with the production `.env` finished and HTTPS working.

```
cd /var/www/nexhris
php artisan config:cache
php artisan route:cache
php artisan view:cache
```

This is a real speed gain and the single most common cause of confusing deployment problems. Understand the rule:

Once `config:cache` has been run, Laravel stops reading `.env` entirely. **Every time you edit `.env` from now on, you must run `php artisan config:clear` — or `config:cache` again — or nothing changes.**

Reload the site and sign in with `admin@ispsec.edu.ph` / `ChangeMe123!`, then change the password immediately from **My Profile**.

Part 9 — Email

The system sends verification codes, password resets, leave notices and announcements. Set it up before you turn two-factor on, because two-factor without working mail locks out the HR Administrator, every Dean and the Campus Director at once.

9.1 Choose a sender

Use the campus mail account if the ICT office will give you SMTP details. A Gmail account works for a demonstration but needs an **App Password**, not the normal password: Google Account → Security → 2-Step Verification → App passwords.

9.2 Set it

```
nano /var/www/nexhris/.env
```

```
MAIL_MAILER=smtp
MAIL_HOST=smtp.gmail.com
MAIL_PORT=587
MAIL_USERNAME=your.address@gmail.com
MAIL_PASSWORD=your-16-character-app-password
MAIL_SCHEME=smtp
MAIL_FROM_ADDRESS="your.address@gmail.com"
MAIL_FROM_NAME="NexHRIS - ISPSC Tagudin"
```

Because the configuration is cached:

```
php artisan config:clear
php artisan config:cache
```

9.3 Prove it works

```
php artisan tinker
```

At the `>>>` prompt:

```
Mail::raw('NexHRIS test from the VPS.', fn ($m) => $m->to('YOUR-OWN-EMAIL@example.com')->subject('NexHRIS test'));
```

Press Enter, then type `exit`. Check your inbox. If nothing arrives:

```
tail -30 /var/www/nexhris/storage/logs/laravel.log
```

`Connection could not be established` almost always means the port is blocked or the app password is wrong.

9.4 Turn two-factor on

Only once a test message has actually arrived:

```
nano /var/www/nexhris/.env      # TWO_FACTOR_ENABLED=true  
php artisan config:clear && php artisan config:cache
```

Sign out and sign in again. You should be asked for a six-digit code, and it should reach your inbox. Two-factor covers the HR Administrator, the Deans and the Campus Director. Employees sign in with a password alone.

Part 10 — Automatic backups

The system writes a single AES-256 encrypted archive containing the database and every uploaded record.

Run one now to prove it works:

```
cd /var/www/nexhris
php artisan backup:run
ls -lh storage/app/backups
```

Schedule it nightly at 2am:

```
crontab -e
```

Choose **1** for nano if asked, then add this line at the end:

```
0 2 * * * cd /var/www/nexhris && /usr/bin/php artisan backup:run >> /var/log/nexhris-
backup.log 2>&1
```

Save and exit.

An encrypted backup sitting on the same server it backs up is not a backup. Download a copy to your own machine regularly. From Windows:

```
scp deploy@YOUR_SERVER_IP:/var/www/nexhris/storage/app/backups/*.zip .
```

Keep **BACKUP_PASSWORD** somewhere other than the server. Without it the archive cannot be opened.

Part 11 — Check the whole system

Work through this list in a browser. Every item should behave as described.

Signing in

- [] `https://` shows a padlock, and `http://` redirects to it
- [] The sign-in page shows the "I agree with the Terms and Conditions" box
- [] Sign in is refused until the box is ticked
- [] "Terms and Conditions" opens the terms, and `/terms` loads on its own
- [] The HR Administrator is asked for a code, and it arrives by email
- [] Five wrong passwords lock the account and the lock is logged

Documents — the part that needs LibreOffice

- [] **Form Templates** → the blank Personal Data Sheet previews as a PDF
- [] **Form Templates** → the blank leave form previews as a PDF with two pages, its tick boxes, both signature blocks and the instructions page
- [] Upload a filled Personal Data Sheet as an employee, then export it — the PDF matches the workbook
- [] A ledger card prints on the official form
- [] **Leave Balances**, **Employee Directory** and the **Leave Calendar** export as PDFs
- [] Every downloaded file is named for the person, not "document.pdf"

Boundaries — prove the privacy claims

- [] A Dean cannot open an employee from another college, even by editing the web address
- [] A Dean cannot reach the create-account screen
- [] Signing out and pasting a `/admin/...` address redirects to sign-in

Errors

- [] A bad address shows the styled 404 page, not a stack trace
- [] `https://nexhris.example.ph/.env` returns 403 or 404 — **never** the file

If any document check fails, LibreOffice is the first suspect:

```
cd /var/www/nexhris
php artisan tinker
>>> app(App\Services\XlsxToPdfService::class)->renderer();
```

It must print `"libreoffice"`. If it prints `"php"`, the binary was not found — check `LIBREOFFICE_PATH` in `.env` and remember to `config:clear` afterwards.

Part 12 — Before the campus uses it

Start with a clean database. The one on your development machine is full of test rows — leave applications reading "testing", ledger lines reading "test", announcements titled "TEST". Part 5.6 and 5.7 give you an empty system with one administrator and the real colleges. Do not copy the development database over.

Create the real accounts. Use campus email addresses, not personal Gmail. Every member of personnel needs a **first day of government service** — it prints in the header of the official leave ledger card and the card is not accepted with it blank.

Publish the official forms. Sign in as the HR Administrator, open **Form Templates**, and upload the current Personal Data Sheet and leave form. Nothing can be filed until these exist.

Tell staff to fill the forms in desktop Excel. Google Sheets, WPS and Excel Online silently discard the tick boxes from Civil Service forms. A form filled in those will upload and convert, but every checkbox will be gone — and that is the file itself, not the conversion.

Updating the system later

From your Windows machine:

```
git add -A && git commit -m "Describe the change" && git push
```

On the server:

```
cd /var/www/nexhris
php artisan down                # show a maintenance page

git pull
composer install --no-dev --optimize-autoloader
npm ci && npm run build && rm -rf node_modules
php artisan migrate --force

php artisan config:cache
php artisan route:cache
php artisan view:cache

php artisan up                  # back online
```

`php artisan down` matters: without it, somebody can hit a half-updated system.

Appendix A — When something breaks

Symptom	Cause	Fix
502 Bad Gateway	PHP-FPM socket name wrong or service stopped	<code>ls /run/php/</code> , correct <code>fastcgi_pass</code> , <code>sudo systemctl restart php8.3-fpm</code>
500, blank page	See the real reason	<code>tail -50 storage/logs/laravel.log</code>
"Permission denied" in the log	<code>storage</code> not writable	<code>sudo chown -R deploy:www-data storage && sudo chmod -R 775 storage</code>
Edited <code>.env</code> , nothing changed	Config is cached	<code>php artisan config:clear && php artisan config:cache</code>
Stylesheet missing, page unstyled	Assets not built	<code>npm ci && npm run build</code>
Blank forms and photos 404	Symlink missing	<code>php artisan storage:link</code>
Leave form downloads as <code>.xlsx</code>	LibreOffice not found	<code>which soffice</code> , set <code>LIBREOFFICE_PATH</code> , <code>config:clear</code>
PDF export times out	Timeouts too short	<code>max_execution_time=180</code> , <code>fastcgi_read_timeout 180</code> , restart both
Nobody with an admin role can sign in	Two-factor on, mail broken	Set <code>TWO_FACTOR_ENABLED=false</code> , <code>config:clear</code> , fix mail, turn it back on
Database connection refused	MariaDB stopped	<code>sudo systemctl status mariadb</code> , <code>sudo systemctl start mariadb</code>

Useful logs:

```
tail -f /var/www/nexhris/storage/logs/laravel.log # the application
tail -f /var/log/nginx/nexhris-error.log         # the web server
sudo journalctl -u php8.3-fpm -n 50              # PHP
```

Appendix B — Uploading without Git

Install WinSCP on Windows, connect by SFTP to `YOUR_SERVER_IP` as `deploy`, and upload the whole `nexhris` folder to `/var/www/`.

Do **not** upload these — they are either rebuilt on the server or must never leave your machine:

- `node_modules/`
- `vendor/`
- `.env`
- `storage/logs/`
- `storage/app/pdf-cache/`

Then return to step 5.2 and carry on.

Appendix C — What each part of the system needs

Useful when something misbehaves and you are narrowing down where to look.

Feature	Depends on
Signing in, two-factor	Database, working SMTP
Personal Data Sheet upload	<code>php-zip</code> , writable <code>storage/app/private</code>
Any form converted to PDF	LibreOffice, <code>proc_open</code> , <code>max_execution_time</code>
Ledger cards and reports	Database only
Blank forms, profile photos	<code>storage:link</code>
Digital ID and QR code	<code>php-gd</code>
Encrypted backups	<code>php-zip</code> , cron, <code>BACKUP_PASSWORD</code>
Leave and PDS notices	SMTP

Nothing in this system uses a queue worker or a scheduled task other than the optional nightly backup, so there is no `queue:work` to keep running.